Project Analysis & Execution Plan: USGS Real-Time Data Pipeline

1. Executive Summary

This project implements an end-to-end distributed data engineering pipeline on macOS (Apple Silicon) using **Colima**, **Docker Compose**, **Spring Boot 3.2.4**, **Apache Kafka 3.7.0**, **Apache Spark 3.5.1**, **Apache Hadoop 3.3.6 (HDFS)**, and **PostgreSQL 16**.

The pipeline ingests real-time streamflow/discharge observations (parameter code `00060`) for Virginia water monitoring stations from the **USGS National Water Information System (NWIS)** REST API: `https://waterservices.usgs.gov/nwis/iv/?format=json&stateCd=va&parameterCd=00060`

2. Architectural Analysis & Critical Findings

A detailed inspection of the repository revealed several functional and architectural blockers preventing end-to-end operation:

1. USGS JSON Schema Mismatch (Major Functional Blocker)

- **Current Behavior**: `UsgsDataIngestionService` pulls the raw endpoint and sends the whole payload (~408 KB string containing ~274 Virginia gauging stations) directly as a single Kafka message under key `"va-water"`.
- **Spark Parser Expectation**: `SparkStreamingProcessor` expects incoming Kafka messages to represent a single observation formatted with top-level fields:

  ```
  json
  {"siteCode": "01484800", "timestamp": "...", "value": 1.80}
  ```

- **Consequence**: Spark evaluates `functions.get_json_object(..., "$.siteCode")` on the complex nested WaterML structure (`value.timeSeries[i]`), which evaluates to `NULL`. Spark writes rows where `site_code`, `timestamp`, and `value` are all null, or causes deserialization failures.

- **Solution**: `UsgsDataIngestionService` must parse `value.timeSeries` via Jackson `ObjectMapper`, extract each station's reading, and produce discrete event records to Kafka keyed by `siteCode`.

2. Empty Kafka Configuration (`KafkaConfig.java`)

- **Current Behavior**: `KafkaConfig.java` is an empty file (0 bytes).
- **Consequence**: The Kafka topic `usgs-water-metrics` is not declared or created by Spring. While Kafka can auto-create topics on first produce if configured, declaring a `NewTopic` bean guarantees proper partition count (e.g., 3 partitions) and replication factor upon application startup.

3. Hadoop DataNode Port Mapping in Docker Compose

- **Current Behavior**: In `docker-compose.yml`, `pipeline-namenode` exposes ports `9870` and `9000`, but `pipeline-datanode` has **no published ports**.
- **Consequence**: In HDFS, clients communicate with the NameNode to initiate a write, but the actual block data transfer is negotiated directly between the client and DataNode on port `9866` (and HTTP port `9864`). When Spark runs on the host (`local[*]`) and attempts to write to `hdfs://localhost:9000`, block creation will fail with `IOException: Could not obtain block` because the DataNode is unreachable from the host.
- **Solution**: Expose ports `9864:9864` and `9866:9866` on the `datanode` container in `docker-compose.yml`, and ensure DataNode hostname resolution is enabled (`dfs.client.use.datanode.hostname=true`).

4. Uninvoked Batch Processing Service

- **Current Behavior**: `SparkBatchProcessor` defines `processKafkaDataToHdfs()`, but it is neither scheduled nor exposed via any REST controller or CLI command.
- **Solution**: Expose a management REST endpoint (`POST /api/batch/hdfs-sync`) and optional scheduled trigger so batch archival to HDFS can be initiated and monitored on demand.

5. Spark Streaming Lifecycle & Error Handling

- **Current Behavior**: `SparkStreamingProcessor` implements `CommandLineRunner`. If Kafka is not reachable when the Spring Boot application boots, the entire Spring context crashes during startup.
- **Solution**: Add connection check / retry resilience or async initialization so the Spring Boot container remains operational even if Kafka is restarting.

6. Gradle Test Configuration & Java 17 `--add-opens`

- **Current Behavior**: `build.gradle` configures the necessary JVM `--add-opens` flags for `bootRun`, but not for the `test` task.
- **Consequence**: Running `./gradlew test` on any test that initializes Spark Catalyst or SparkSession will trigger Java 17 module encapsulation `IllegalAccessError`.
- **Solution**: Extract common JVM arguments in `build.gradle` and apply them to both `bootRun` and `test`.

3. Step-by-Step Execution Plan

Phase 1: Infrastructure & Configuration Refinement

1. **Enhance `docker-compose.yml`**:
    - Add port mappings to `datanode`: `"9864:9864"` and `"9866:9866"`.
    - Ensure HDFS configuration includes `dfs.client.use.datanode.hostname = true` so the macOS host can communicate with DataNode.
2. **Populate `KafkaConfig.java`**:
    - Define a Spring `@Configuration` bean creating `NewTopic("usgs-water-metrics", 3, (short) 1)`.

Phase 2: Ingestion Service Refactoring

1. **Refactor `UsgsDataIngestionService.java`**:
    - Implement structured parsing using Jackson `JsonNode`:
        - Traverse `rootNode.path("value").path("timeSeries")`.
        - For each station, extract `siteCode` (e.g., `"01484800"`), `siteName`, `dateTime`, and numerical `value`.
        - Handle provisional or missing values (e.g., `-999999.0`).
    - Publish individual JSON records to Kafka topic `usgs-water-metrics` using `siteCode` as message key.
    - Maintain the `@Scheduled` polling interval with structured logging.

Phase 3: Spark Streaming & Persistence Optimization

1. **Refactor `SparkStreamingProcessor.java`**:

- Update schema definition to cleanly match the emitted JSON events.
- Add watermark and checkpointing directory management.
- Ensure the PostgreSQL JDBC sink properly inserts into `water_metrics` table without conflicting with Hibernate's sequence generator.

2. **Wire `SparkBatchProcessor.java`**:
   - Connect `SparkBatchProcessor` to a REST trigger in `ReportingController` (`POST /api/batch/hdfs-sync`).
   - Support writing structured JSON / Parquet to `hdfs://localhost:9000/data/usgs_water_metrics`.

Phase 4: Build, Testing & Quality Assurance

1. **Update `build.gradle`**:
   - Add shared JVM `--add-opens` to `tasks.named('test')`.
2. **Add Unit & Integration Tests**:
   - Add tests for USGS JSON parsing logic under `src/test/java/com/pipeline/ingestion/`.
   - Add tests for `ReportingController` under `src/test/java/com/pipeline/reporting/`.

Phase 5: End-to-End Verification

1. Start Colima (`colima start --cpu 4 --memory 8`).
2. Start Docker Compose services (`docker compose up -d`).
3. Boot the application (`./gradlew bootRun`).
4. Verify Kafka messages via Kafka console consumer.
5. Verify PostgreSQL table `water_metrics` populated with streamflow records.
6. Verify REST endpoint `GET http://localhost:8081/api/reports/latest-water-metrics`.
7. Trigger `POST http://localhost:8081/api/batch/hdfs-sync` and verify files in HDFS via NameNode Web UI (`http://localhost:9870`).

4. Verification Checklist

Step	Target Component	Verification Method
1	Docker / Colima	`docker compose ps` shows all 5 containers `Up`
2	Kafka Topic	`kafka-topics.sh --bootstrap-server localhost:9092 --list` includes `usgs-water-metrics`
3	Ingestion	Application logs confirm: `Published N water metric records to Kafka`
4	Spark Streaming	Logs show `=== PROCESSING BATCH N ===` with non-null values
5	PostgreSQL	`SELECT COUNT(*), AVG(value) FROM water_metrics;` returns valid metrics
6	REST API	`curl -s http://localhost:8081/api/reports/latest-water-metrics` returns JSON array
7	HDFS Sync	`hdfs dfs -ls /data` shows exported batch files